



CS-2001
Data Structures
Fall 2023

Graph Algorithm (MST, SSSP)

Rizwan Ul Haq

Assistant Professor

FAST-NU

rizwan.haq@nu.edu.pk

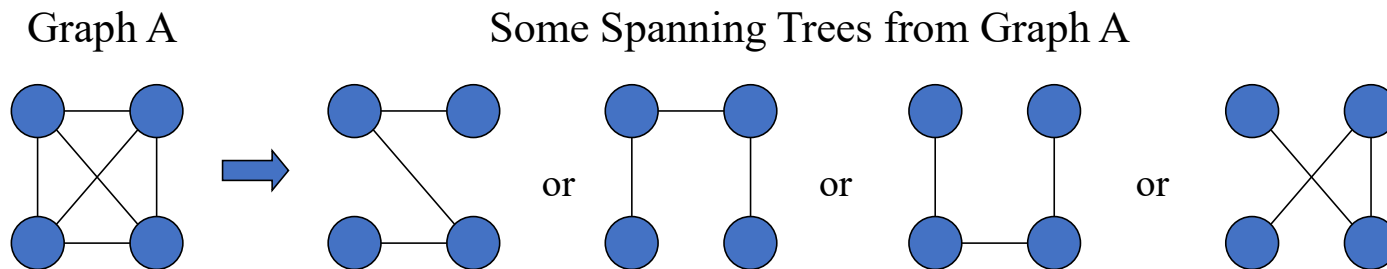
Minimum Spanning Tree

- A **tree** is a **connected graph** without cycles
- A **connected graph** is a **tree** if and only if it has N vertices and $N-1$ edges
- A subgraph that spans (reaches out to) all vertices of a graph is called a **spanning subgraph**
- A subgraph that is a tree and that spans (reaches out to) all vertices of the original graph is called a **spanning tree**
- Among all the spanning trees of a weighted and connected graph, the one (possibly more) with the least total weight is called a **minimum spanning tree (MST)**

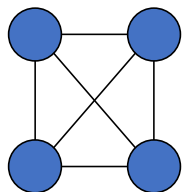
Spanning Trees

A spanning tree of a graph is just a subgraph that contains all the vertices and is a tree.

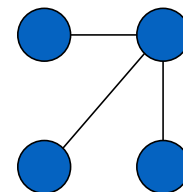
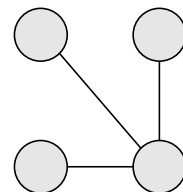
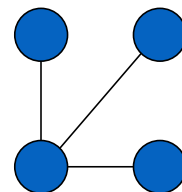
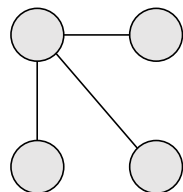
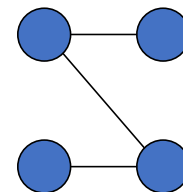
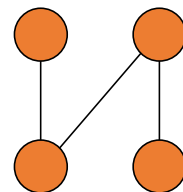
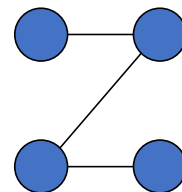
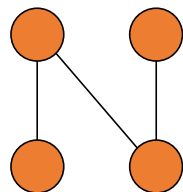
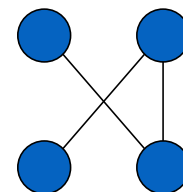
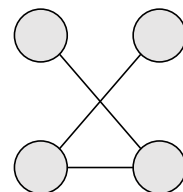
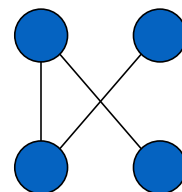
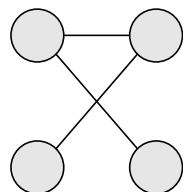
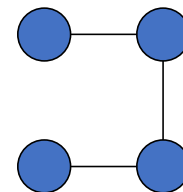
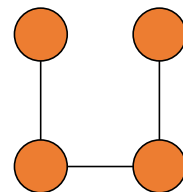
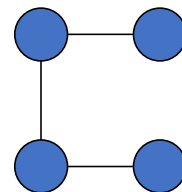
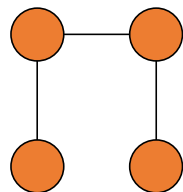
A graph may have many spanning trees.



Complete Graph



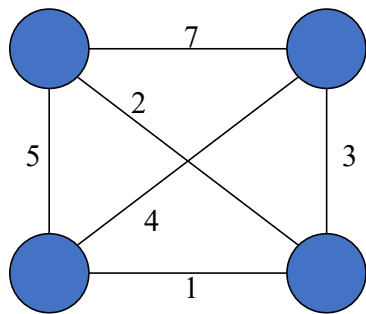
All 16 of its Spanning Trees



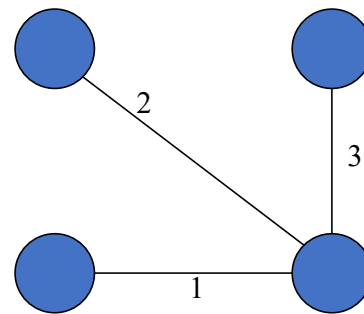
Minimum Spanning Trees

The Minimum Spanning Tree for a given graph is the Spanning Tree of minimum cost for that graph.

Complete Graph



Minimum Spanning Tree



Minimum Spanning Tree

- Is MST Unique???
 - Yes if edge lengths are distinct
 - No Otherwise
- In MST there is a single path between any two vertices??? Is this true.
 - Yes because there is no cycle

Algorithms for Obtaining the Minimum Spanning Tree

- Prim's Algorithm
- Kruskal's Algorithm

Prim's Algorithm

- This algorithm starts with one node
- It then, one by one, adds a node that is unconnected to the new graph
- Each time selecting the node whose connecting edge has the smallest weight out of the available nodes' connecting edges

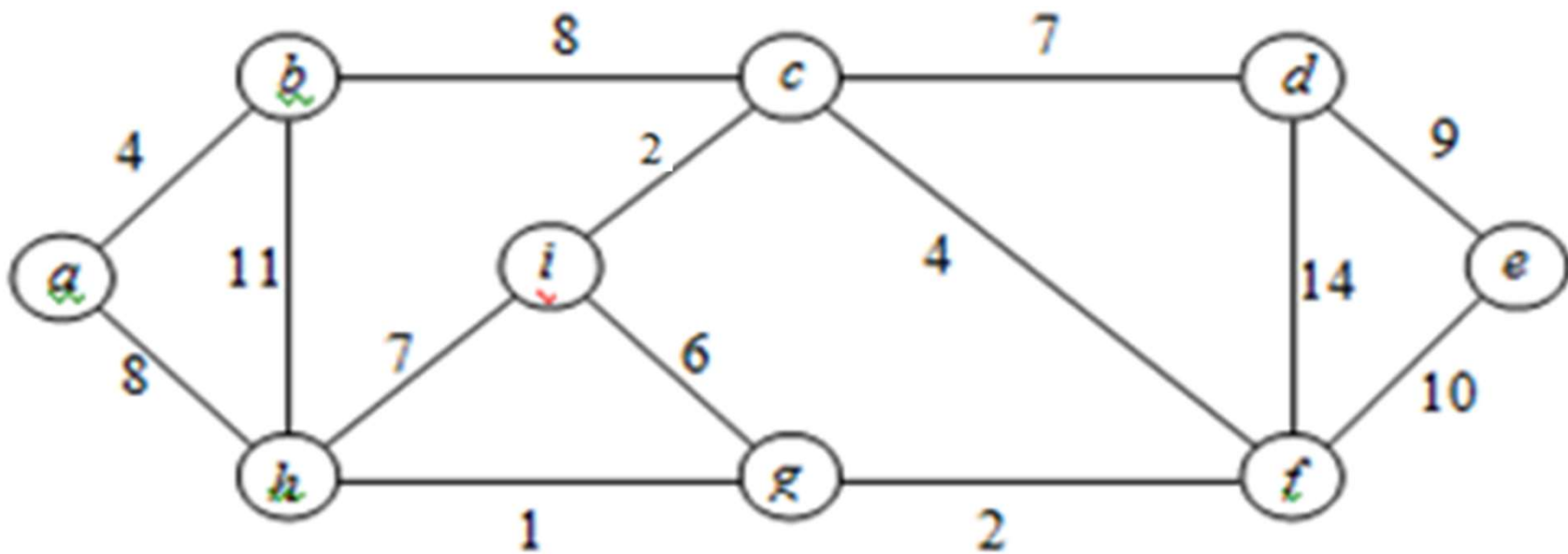
Steps:

- We now start the iterations.
- We select the node that has the least distance, say 'u'.
 - In the first loop, the source is selected because its distance is zero
 - In subsequent iterations this process ensures that the minimum weight edge is selected. We now include the node in the tree.
- Then we relax all the nodes connected to the node
 - This means that we see for each node v connected to u whether the distance of v is greater than the weight of edge (u,v)
 - If it is, we assign $w(u,v)$ to the distance of v
 - This step ensures that after each new node is added to the tree, the distance of the remaining nodes from the tree is updated
- Thus when we have all the edges in the tree, our minimum spanning tree is ready

PRIM'S Algorithm

```
1. PRIM( $G$ ,  $w$ ,  $r$ )
2. for each  $u$  belongs to  $V[G]$ 
3.   do  $key[u] \leftarrow \infty$ 
4.    $\pi[u] \leftarrow \text{NIL}$  (Predecessor)
5.  $key[r] \leftarrow 0$ 
6.  $Q \leftarrow V[G]$  // build min heap
7. while  $Q \neq \emptyset$ 
8.   do  $u \leftarrow \text{EXTRACT-MIN}(Q)$ 
9.   for each  $v$  belongs to  $\text{Adj}[u]$ 
10.    do if  $v$  belongs to  $Q$  and  $w(u, v) < key[v]$ 
11.      then  $\pi[v] \leftarrow u$ 
12.       $key[v] \leftarrow w(u, v)$ 
// modify node and maintain heap property
```

Prim's Algorithm Example



Execution of Prim's Algorithm

For Loop

$V\{G\} = \{a, b, c, d, e, f, g, h, i\}$

$\text{key}[a] = \infty, \text{key}[b] = \infty, \text{key}[c] = \infty$

$\text{key}[d] = \infty, \text{key}[e] = \infty, \text{key}[f] = \infty$

$\text{key}[g] = \infty, \text{key}[h] = \infty, \text{key}[i] = \infty$

As no vertex has
been connected to
tree

$\text{parent of } [a] = \text{Nil}, \text{parent of } [b] = \text{Nil},$

$\text{parent of } [c] = \text{Nil}, \text{parent of } [d] = \text{N}$

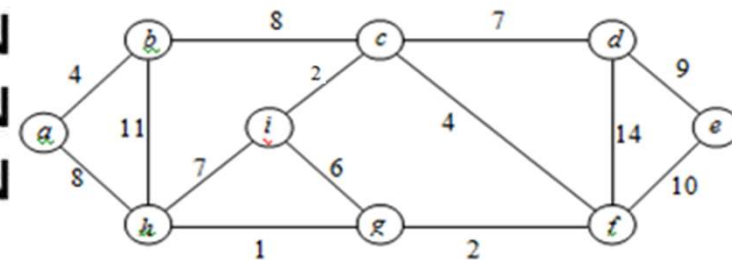
$\text{parent of } [e] = \text{Nil}, \text{parent of } [f] = \text{N}$

$\text{parent of } [g] = \text{Nil}, \text{parent of } [h] = \text{N}$

$\text{parent of } [i] = \text{Nil},$

$\text{key}[a] = 0$

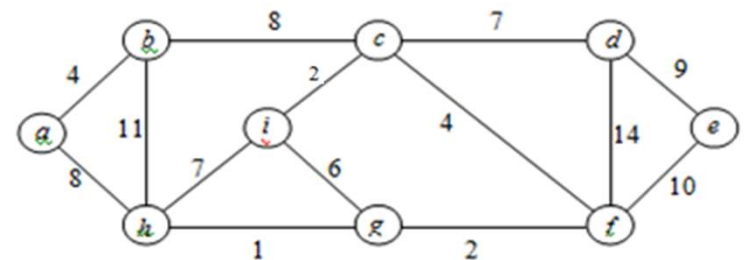
Vertices in $G = Q = \{a, b, c, d, e, f, g, h, i\}$

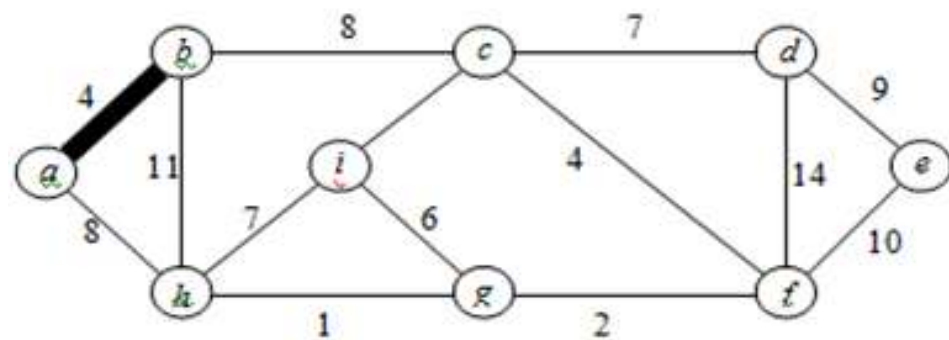
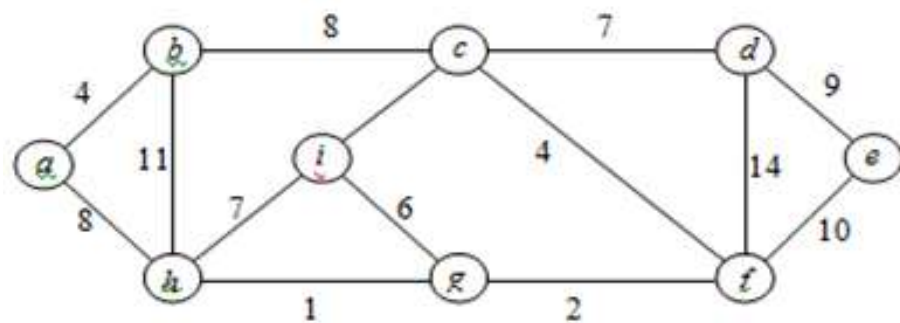


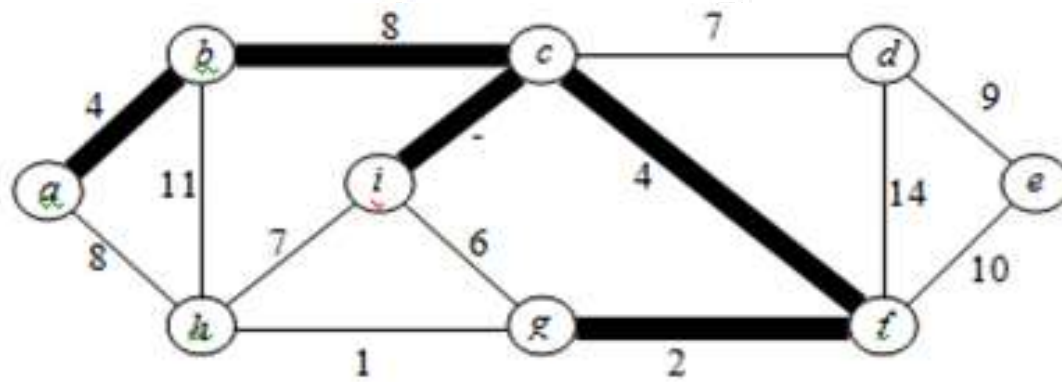
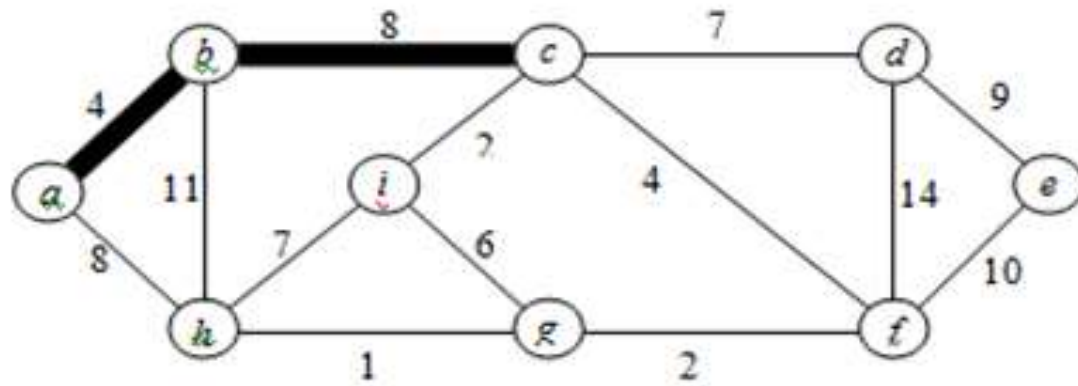
while Q != $\emptyset$		<i>Extract</i> <i>u</i>	Adj {u} = v in Q	For each V $\in$ Adj <i>u</i>		
				if	$\Pi \{v\}$	key {v}
1	T	a	b, h	T, T	a	key[b] = 4, key[h] = 8
2	T	b	c, h	T, F	b	key[c] = 8
3	T	c	d, f, i	T, T, T	c	key[d] = 7, key[f] = 4, key[i] = 2
4	T	i	g, h	T, T	i	key[g] = 6, key[h] = 7
5	T	f	d, g, e	F, T, T	f	key[e] = 10, key[g] = 2
6	T	h	T	T	g	key[h] = 1
7	T	h	-		-	-
8	T	d	e	T	d	key[e] = 9
9	T	e	-	-	-	-

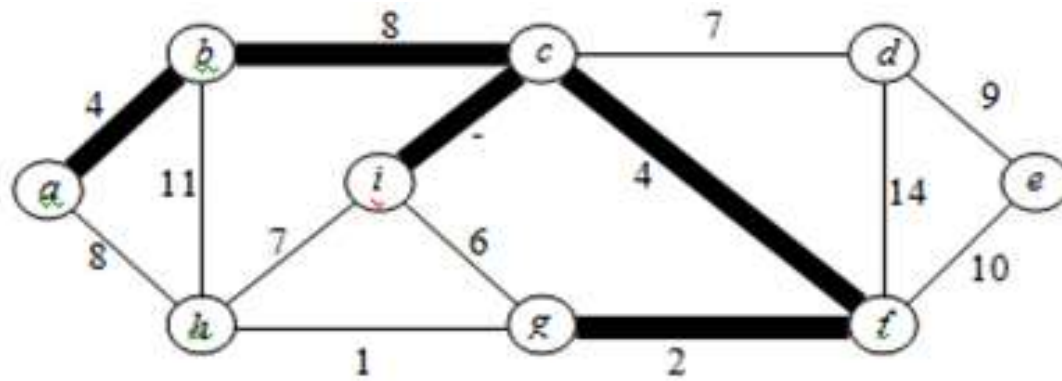
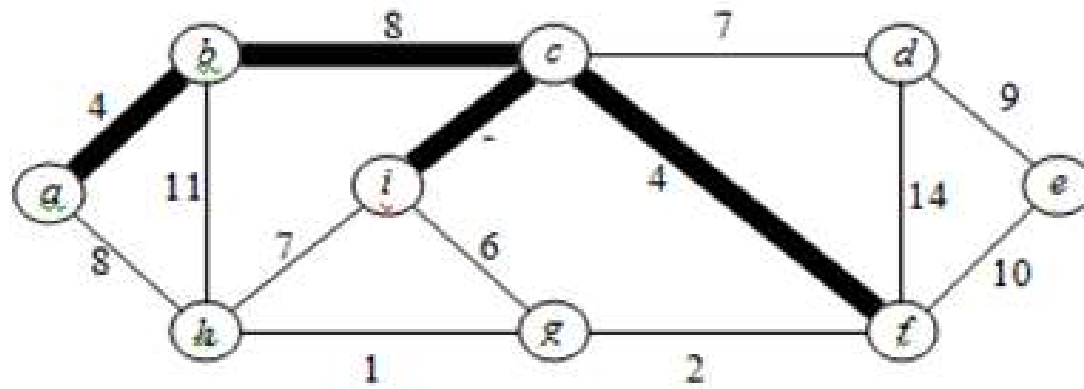
Removing u from set Q

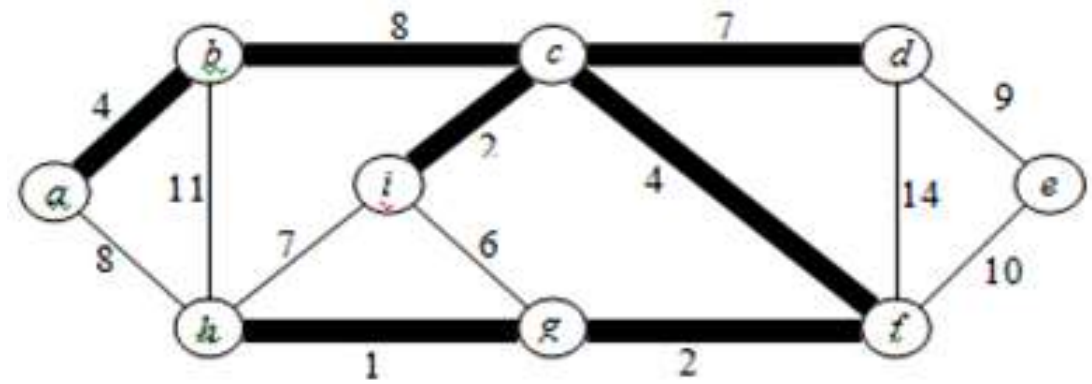
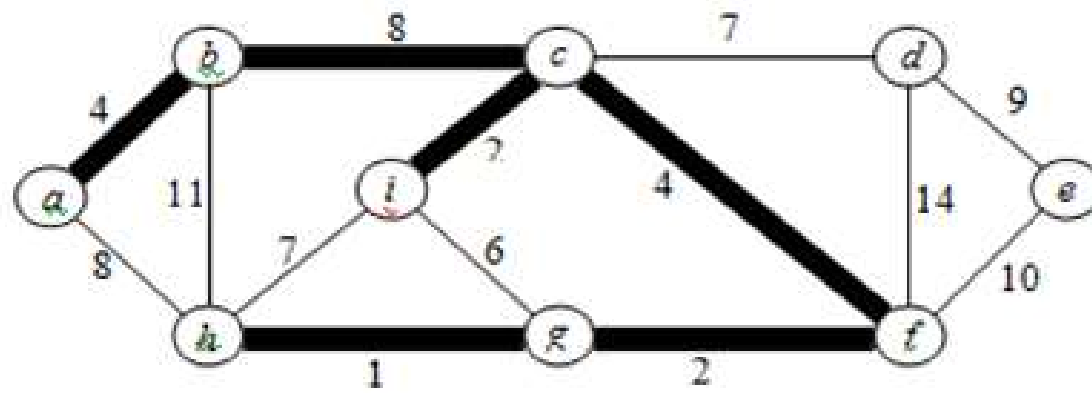
- Add it to the set = vertices in the MST= {a,b,c,i,f,g,h,d,e}
- Thus adding ($\Pi\{u\}, u$) edge to A

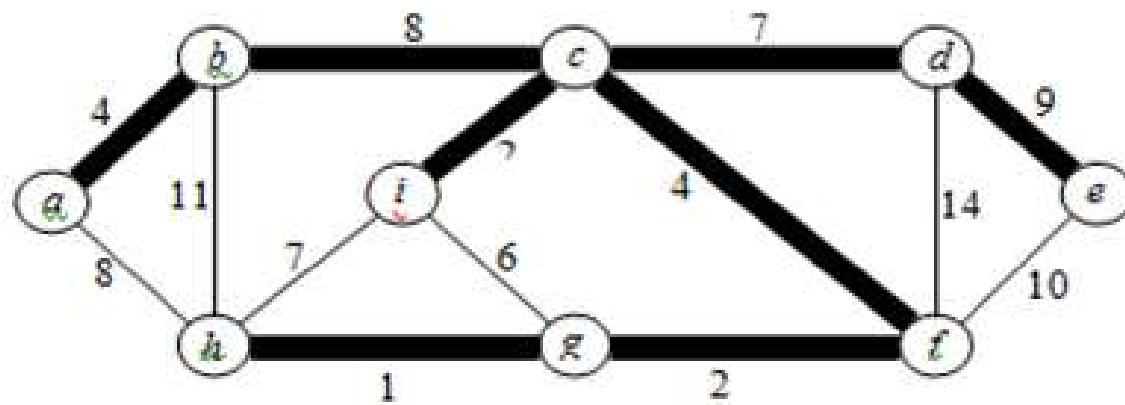












Safe Edges

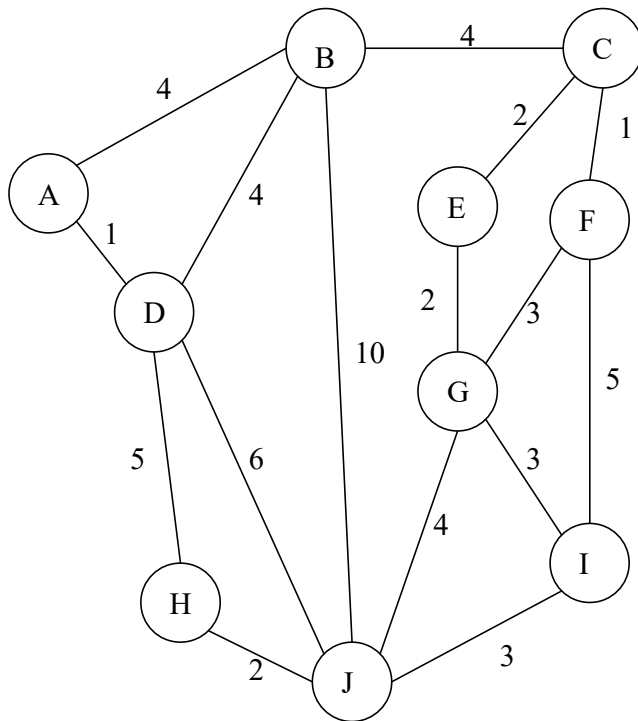
Analysis

- $O(E \log V)$
 - 1-4 $\Rightarrow O(V)$
 - 7-10 $\Rightarrow (E \log V + V \log V)$
- Prim's algorithm doesn't need to worry if adding an edge will create a cycle. Since this algorithm deals primarily with the nodes, and not the edges
- Prim's algorithm is significantly faster in the limit when you've got a really dense graph with many more edges than vertices
- Prim's is faster than Kruskal's in the case of complex graphs.

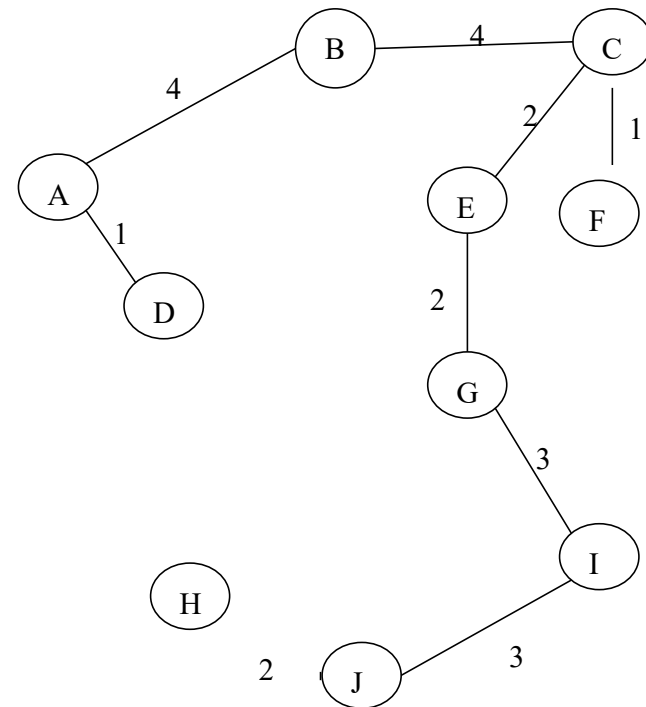
```
1. PRIM(G, w, r)
2. for each u belongs to V[G]
3. do key[u]  $\leftarrow \infty$ 
4.  $\pi[u] \leftarrow \text{NIL}$  (Predecessor)
5. key[r]  $\leftarrow 0$ 
6. Q  $\leftarrow V[G]$  // build min heap
7. while Q  $\neq \emptyset$ 
8. do u  $\leftarrow \text{EXTRACT-MIN}(Q)$ 
9. for each v belongs to Adj[u]
10. do if v belongs to Q and
      w(u, v) < key[v]
11. then  $\pi[v] \leftarrow u$ 
12. key[v]  $\leftarrow w(u, v)$ 
// modify node and maintain heap
property
```

Task

Complete Graph



MST



Home Task

